Introduction to Quantum Database Principle

Xiaofan Li

School of Computer Science, Nanjing University

Different Models, the Same Classical Bits

Consider a collection of research papers. What do we want to ask?

RELATIONAL DATABASE

Rows and columns
`Paper(id, title, year)`
Which papers are from 2025?

VECTOR DATABASE

Embedding vectors
$p_1 \mapsto (0.2, 0.8, \ldots)$
Which papers are similar to this one?

GRAPH DATABASE

Nodes and edges
$p_1 \xrightarrow{\text{cites}} p_2$
Which papers cite this paper?

Different ways to organize and query data; all typically encoded as **bits**.

Data Model vs. Information Representation

DATA MODEL: HOW DO WE ORGANIZE INFORMATION?

Tables, vectors, and graphs describe the structure visible to applications.

INFORMATION REPRESENTATION: WHAT CARRIES THE INFORMATION?

Classical bit
A definite 0 or 1. Text, identifiers, and vector coordinates are encoded using strings of bits.

Quantum bit (qubit)
A state can be $\alpha|0\rangle + \beta|1\rangle$. Measuring in this basis gives 0 or 1, not both values to read out.

A vector of numbers in a database is still classical data; it is not automatically a quantum state.

Reading: IBM Quantum Learning, Basics of Quantum Information: Single Systems.

What Do We Mean by Quantum Data?

Quantum data is information carried by quantum states. There are two useful starting points.

CLASSICAL DATA, QUANTUM PROCESSING

Start with ordinary records or candidate labels. Prepare a quantum encoding, apply an algorithm, then read a classical result.

DATA THAT IS ALREADY QUANTUM

Start with states produced by a quantum experiment or simulation. Measurement extracts information and can change the state.

In this lecture, we focus on the first route: can quantum processing help a database task? This changes the computation, not necessarily the logical data model.

Biamonte et al., Quantum Machine Learning, Nature 2017; IBM Quantum Learning: General Measurements.

Why Consider Quantum Databases?

Start with a basic database task: **find a record satisfying a condition**.

THE UNSTRUCTURED CASE: NO SHORTCUT TO THE ANSWER

Records have **no useful ordering, index, or other structure** that helps locate a match. We can only test candidates against the condition.

For one matching record among N candidates, classical search needs $\Theta(N)$ tests in the worst case. Grover search uses $O(\sqrt{N})$ oracle calls with high success probability.

This is a hard baseline for search: even when classical search cannot exploit structure, quantum computation can reduce the number of tests. Total runtime also depends on access and oracle costs.

Reading: Grover, A Fast Quantum Mechanical Algorithm for Database Search, STOC 1996.

Outline

MOON

CAT

EVOLVE

APPLE

Annealing

END

Outline

MOON

CAT

EVOLVE

APPLE

Annealing

END

MOON

One Question about the Moon



When you are not looking at the moon, is it there?

Outline

MOON

CAT

EVOLVE

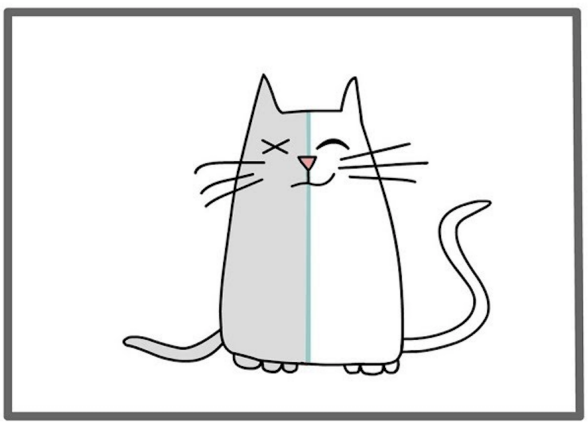
APPLE

Annealing

END

CAT

Schrödinger's Cat



Electron: if we observe an electron, it will be in either of the two state: spin up, or spin down.

Schrödinger's Cat: There is a cat, a gun, and an electron, in a black box. If the electron spins down, the gun will automatically kill the cat; if the electron spins up, the gun will do nothing. If we open the box and observe, the cat will definitely either be alive or dead. The question is: if we don't open the box, is the poor cat alive or dead?

State

Classical state:

- Cat state: $|alive\rangle$ or $|dead\rangle$
- Bit state: $|0\rangle$ or $|1\rangle$
- Die state: $|1\rangle$, $|2\rangle$, $|3\rangle$, $|4\rangle$, $|5\rangle$, $|6\rangle$

Quantum state, when we are observing it:

- Electron state: $|up\rangle$ or $|down\rangle$
- Schrödinger's cat state: $|alive\rangle$ or $|dead\rangle$

What's the difference?

The difference shows up when we are not looking at the state.

Superposition: what is a state like when we are not observing it?

Classical state:

- Cat state: $|alive\rangle$ or $|dead\rangle$
- Bit state: $|0\rangle$ or $|1\rangle$
- Die state: $|1\rangle$, $|2\rangle$, $|3\rangle$, $|4\rangle$, $|5\rangle$, $|6\rangle$

Quantum state, when we are not observing it:

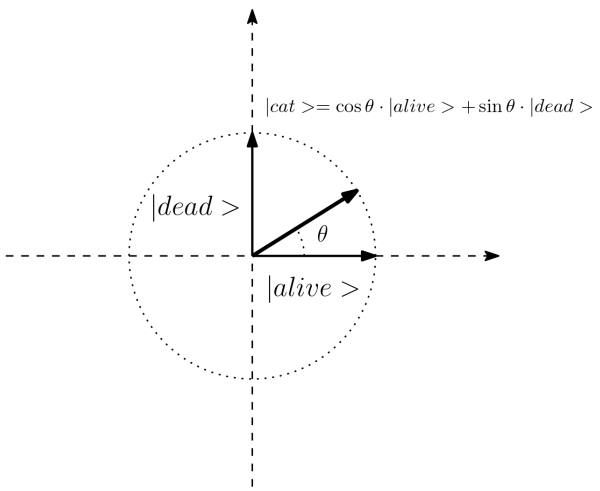
- Electron state: $|up\rangle + |down\rangle$
- Schrödinger's cat state: $|alive\rangle + |dead\rangle$

Note 1: Superposition state is a new state. It is neither $|alive\rangle$ nor $|dead\rangle$. It is $|alive\rangle + |dead\rangle$.

Note 2: The $+$ is vector addition. The displayed sums omit a normalization factor of $1/\sqrt{2}$.

Note 3: $|alive\rangle + |dead\rangle$ means that if we observe the cat, it will **randomly collapse** into either $|alive\rangle$ or $|dead\rangle$. But if we don't observe, it is neither of $|alive\rangle$ or $|dead\rangle$.

Bohr: God rolls a die.



$$|cat\rangle = \cos\theta \cdot |alive\rangle + \sin\theta \cdot |dead\rangle$$
$$P_{alive} = \cos^2\theta$$
$$P_{dead} = \sin^2\theta$$

State vector

For the real-amplitude example, we can rewrite $|cat\rangle$ using a vector form:

$$|alive\rangle \longrightarrow \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \qquad |dead\rangle \longrightarrow \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

$$|cat\rangle = \cos\theta \begin{bmatrix} 1 \\ 0 \end{bmatrix} + \sin\theta \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} \cos\theta \\ \sin\theta \end{bmatrix}.$$

Outline

MOON

CAT

EVOLVE

APPLE

Annealing

END

EVOLVE

What is evolving?

Evolve: if a state $|A\rangle$ transits to a new state $|B\rangle$, we say it **evolves** from $|A\rangle$ to $|B\rangle$.

For example,

- if someone kills his cat, then $|cat\rangle$ will evolve from $|alive\rangle$ to $|dead\rangle$
- if I use a NOT gate to act on a bit $|0\rangle$, then $|0\rangle$ will evolve from $|0\rangle$ to $|1\rangle$
- If I push a stone, then $|stone\rangle$ will evolve from $|static\rangle$ to $|moving\rangle$

Mechanics: if we know a state, and we also know how it evolves, then we know **mechanics** of it.

$$\text{State} + \text{Evolving Rules} = \text{Mechanics}$$

For example: cat killer mechanics,

information mechanics, and,

Newton/Classical Mechanics

Have you notice the Newton mechanics?

In Newton/classical mechanics,

- the state of an object is a triplet: $|position, velocity, mass\rangle$,
- the evolving rule is Newton's second law: $F = ma$

$$|position, velocity, mass\rangle + \text{Newton's second law} = \text{Newton Mechanics}$$

For example, if we know the triplet state of a GPS satellite, we will definitely predict the state of it after one hour. This is because how a satellite state evolves follows the Newton's second law, and scientists know this evolving rule very well.

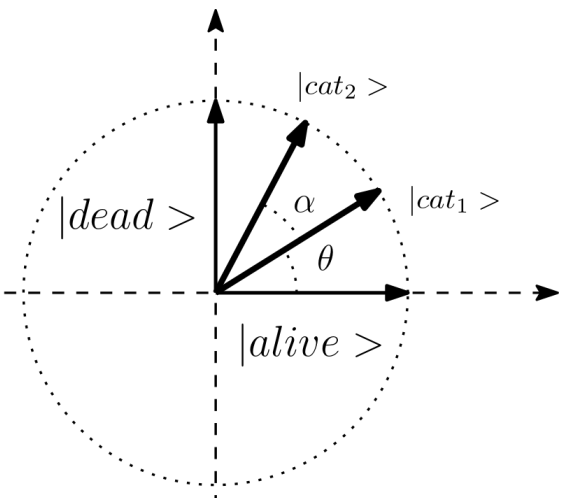
How a quantum Schrödinger's cat state evolves?

Quantum state $|cat\rangle$ evolves according to Schrödinger's equation:

$$-i\hbar\frac{\partial}{\partial t}|cat\rangle = (\frac{\hbar^2}{2m}\bigtriangledown^2 - V)|cat\rangle$$

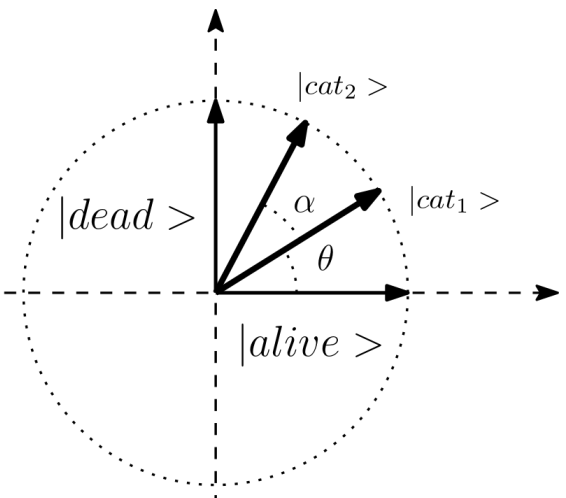
A quantum state evolves according to Schrödinger's equation, just like a classical triplet state evolves according to Newton second's law.

Quantum state evolves by vector rotation



- $|cat\rangle$ is a unit vector on the unit circle
- How to evolve a unit vector to another unit vector on the same unit circle?
- Yes, by **vector rotation**!
- How to do vector rotation?

Yes, by matrix multiplication!

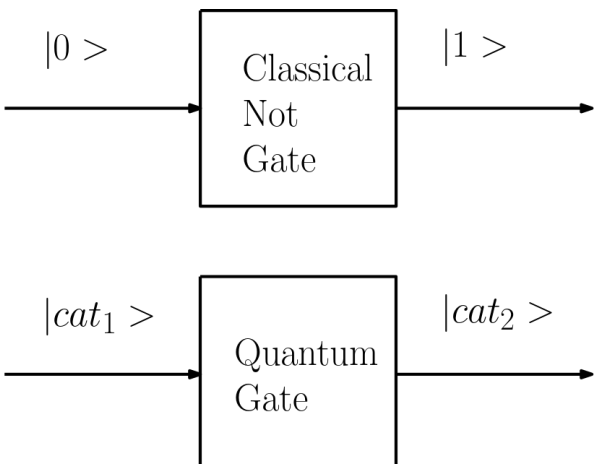


$$|cat_1\rangle = \begin{bmatrix}\cos\theta \\ \sin\theta\end{bmatrix}, |cat_2\rangle = \begin{bmatrix}\cos(\theta+\alpha) \\ \sin(\theta+\alpha)\end{bmatrix}$$

$$|cat_2\rangle = \begin{bmatrix}\cos\alpha & -\sin\alpha \\ \sin\alpha & \cos\alpha\end{bmatrix} \cdot |cat_1\rangle$$

The rotation matrix rotates a unit vector with angle θ to another unit vector with angle $\theta + \alpha$ counterclockwise.

Quantum Gate



For our two-state, real-amplitude example, the evolution over a chosen interval can be represented by a simple 2×2 rotation matrix.

$$-i\hbar\frac{\partial}{\partial t}\left|cat\right\rangle = (\frac{\hbar^2}{2m}\bigtriangledown^2 - V)\left|cat\right\rangle$$

$$\rightsquigarrow \quad \begin{bmatrix} \cos\alpha & -\sin\alpha \\ \sin\alpha & \cos\alpha \end{bmatrix}$$

Now, what is quantum computation?

Just like we explain mechanics as:

$$\text{Mechanics} = \text{State} + \text{Evolve}$$

The quantum computation could be understand as:

$$\text{Quantum Computation} = \text{Qubit} + \text{Quantum Gate}$$

Outline

MOON

CAT

EVOLVE

APPLE

Annealing

END

APPLE

Delicious Apple Searching Problem

Delicious Apple Searching Problem: There are N apples that look same. Only one of them is a sweet delicious apple (a good apple), while all the remaining $N - 1$ apples taste very bitter (bad apples). In the worst case how many times you have to try to find the good apple?

Note:

- No parallel method is allowed, i.e., you can't ask your friends to taste apples simultaneously
- We count the times in the worst case (worst-case time complexity)
- Unstructured database searching

One minute to think about this problem.

Classical Answer

Classical search requires $\Theta(N)$ **tests** in the worst case to find the good apple.

Quantum Algorithm: Grover's Search

States: $|good\rangle, |bad_1\rangle, |bad_2\rangle, \ldots, |bad_{N-1}\rangle$.

EQUAL SUPERPOSITION STATE

$$|apple\rangle = \frac{1}{\sqrt{N}}\Big(|good\rangle + |bad_1\rangle + \cdots + |bad_{N-1}\rangle\Big)$$

$$= \frac{1}{\sqrt{N}}|good\rangle + \frac{1}{\sqrt{N}}\sum_{i=1}^{N-1}|bad_i\rangle$$

Every apple has the same amplitude $1/\sqrt{N}$, hence the same measurement probability $1/N$.

Quantum Algorithm: Grover's Search

Now define two normalized vectors, exactly as before:

$$|GOOD\rangle = |good\rangle\,, \qquad |BAD\rangle = \frac{1}{\sqrt{N-1}} \sum_{i=1}^{N-1} |bad_i\rangle$$

Thus $\sum_{i=1}^{N-1} |bad_i\rangle = \sqrt{N-1}\,|BAD\rangle$. Substitute this into the previous expression:

$$|apple\rangle = \frac{1}{\sqrt{N}}\,|GOOD\rangle + \sqrt{\frac{N-1}{N}}\,|BAD\rangle$$

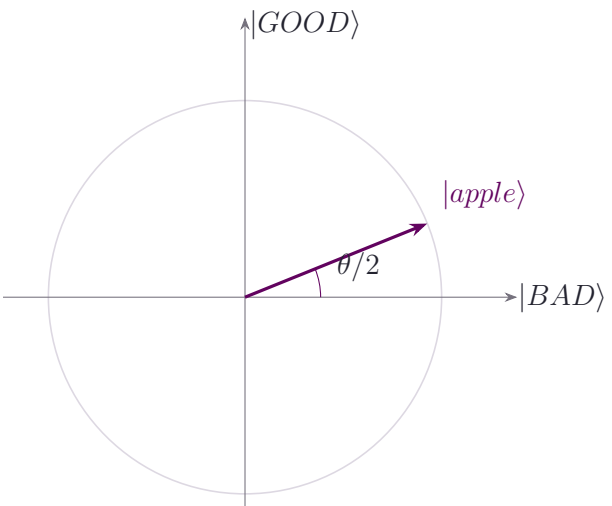
GOOD represents the one delicious apple; BAD combines all the bitter apples into one direction.

Quantum Algorithm: Grover's Search

$$\cos\frac{\theta}{2} = \sqrt{\frac{N-1}{N}}, \qquad \sin\frac{\theta}{2} = \frac{1}{\sqrt{N}}$$

$$|apple\rangle = \cos\frac{\theta}{2}|BAD\rangle + \sin\frac{\theta}{2}|GOOD\rangle$$

For large N, $\theta/2$ is small. The initial vector is close to BAD, so an immediate observation is very likely to give a bitter apple.



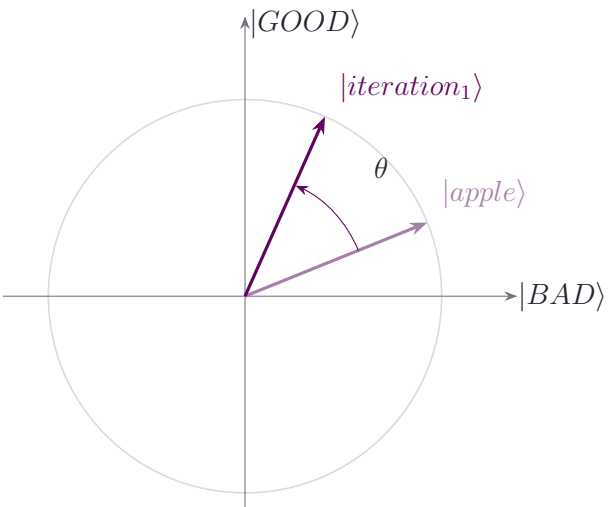
Quantum Algorithm: Grover's Search

How can we evolve $|apple\rangle$ toward $|GOOD\rangle$? Yes, by rotation.

One Grover iteration rotates the vector counterclockwise by θ:

$$\frac{\theta}{2} \quad \longrightarrow \quad \frac{\theta}{2} + \theta = \frac{3\theta}{2}$$

Repeat the same rotation. Stop when the vector is close to GOOD; then observe it.



Quantum Algorithm: Grover's Search

The initial angle is $\theta/2$. Each iteration adds θ.

$$|apple\rangle = \cos\frac{\theta}{2}|BAD\rangle + \sin\frac{\theta}{2}|GOOD\rangle$$

$$|iteration_1\rangle = \cos\frac{3\theta}{2}|BAD\rangle + \sin\frac{3\theta}{2}|GOOD\rangle$$

$$|iteration_k\rangle = \cos\frac{(2k+1)\theta}{2}|BAD\rangle + \sin\frac{(2k+1)\theta}{2}|GOOD\rangle$$

After k rotations: $\theta/2 + k\theta = (2k+1)\theta/2$. The GOOD probability is the square of its coefficient.

Quantum Algorithm: Grover's Search

We want the final vector to be close to GOOD, whose angle is $\pi/2$.

$$\frac{(2k+1)\theta}{2} \approx \frac{\pi}{2}$$

$$2k+1 \approx \frac{\pi}{\theta} \qquad \Longrightarrow \qquad k \approx \frac{\pi}{2\theta} - \frac{1}{2}$$

Choose a nearby integer k. The final angle is usually close to, rather than exactly, $\pi/2$.

Quantum Algorithm: Grover's Search

From the initial state, $\sin(\theta/2) = 1/\sqrt{N}$. For large N, use $\sin u \approx u$ (in radians):

$$\frac{\theta}{2} \approx \frac{1}{\sqrt{N}} \qquad \Longrightarrow \qquad \theta \approx \frac{2}{\sqrt{N}}$$

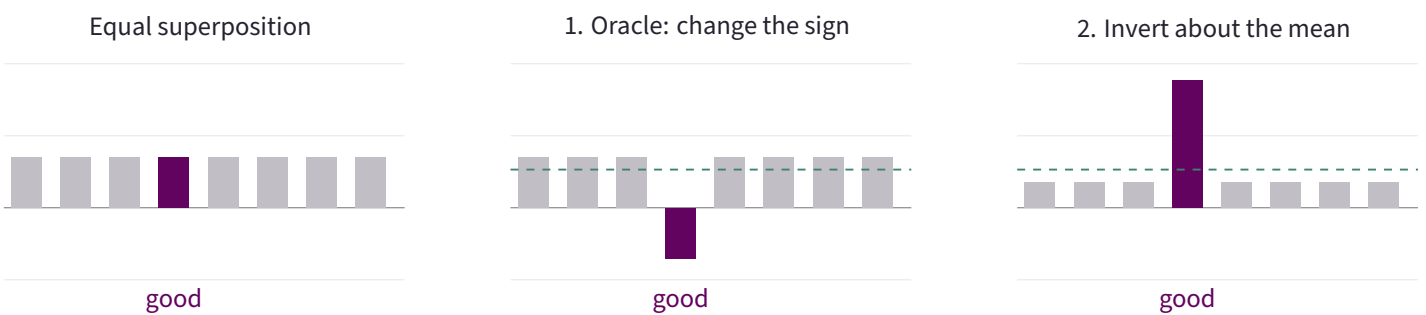
$$k \approx \frac{\pi}{2\theta} - \frac{1}{2} \approx \frac{\pi}{4}\sqrt{N} - \frac{1}{2} = O(\sqrt{N})$$

After $O(\sqrt{N})$ rotations, observe the state: it gives the delicious apple with high probability. This counts search iterations (each using one test), not data-loading cost.

Grover Search: An Alternative Derivation

An alternative derivation through two amplitude flips. Eight states are listed in the same order as the bars.

$$|apple\rangle = \frac{1}{\sqrt{8}}\left(|bad_1\rangle + |bad_2\rangle + |bad_3\rangle + |good\rangle + |bad_4\rangle + \cdots + |bad_7\rangle\right)$$



Purple marks the good apple. The dashed line is the mean after marking; the second flip reflects every amplitude about this line.

Step 1: Mark the Good Apple

Start with eight equal amplitudes, each $a = 1/\sqrt{8}$.

ORACLE: CHANGE THE SIGN

$$a_{good} : a \longmapsto -a$$

$$a_{bad} : a \longmapsto a$$

MEAN AFTER MARKING

$$\mu = \frac{7a - a}{8} = \frac{3}{4}a$$

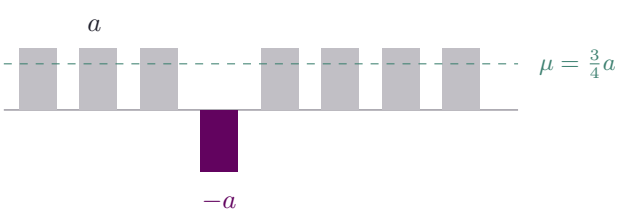
Seven positive amplitudes and one negative amplitude are averaged.

Marking alone does not change probabilities: $|-a|^2 = |a|^2 = 1/8$. It prepares the next step.

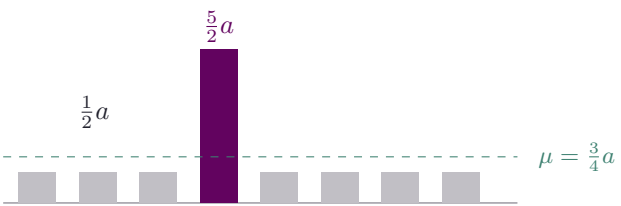
Step 2: Invert About the Mean

$a = 1/\sqrt{8}$, $\mu = 3a/4$. Reflect each amplitude: $a_i \mapsto 2\mu - a_i$.

BEFORE: AFTER THE ORACLE



AFTER: REFLECT ABOUT THE MEAN



Good: $2\mu - (-a) = \frac{5}{2}a$

Each bad: $2\mu - a = \frac{1}{2}a$

Good probability: $25/32 = 0.78125$; each bad: $1/32 = 0.03125$. Total: $25/32 + 7(1/32) = 1$.

From Two Flips to a Recurrence

One good apple: g_k is its amplitude; b_k is the amplitude of each bad apple. Initially, $g_0 = b_0 = 1/\sqrt{N}$. Let $h_k = \sqrt{N-1}\, b_k$ to simplify the algebra ($N > 1$).

1. SUBSTITUTE THE MEAN

$$\mu_k = \frac{(N-1)b_k - g_k}{N}$$

$$g_{k+1} = 2\mu_k + g_k$$

$$= \left(1 - \frac{2}{N}\right) g_k + \frac{2(N-1)}{N} b_k$$

$$b_{k+1} = -\frac{2}{N} g_k + \left(1 - \frac{2}{N}\right) b_k$$

2. REWRITE THE COEFFICIENTS

$$\sin\alpha = \frac{1}{\sqrt{N}}, \quad \cos\alpha = \frac{\sqrt{N-1}}{\sqrt{N}}$$

$$\cos 2\alpha = 1 - \frac{2}{N}, \quad \sin 2\alpha = \frac{2\sqrt{N-1}}{N}$$

$$g_{k+1} = \cos 2\alpha\, g_k + \sin 2\alpha\, h_k$$

$$h_{k+1} = \cos 2\alpha\, h_k - \sin 2\alpha\, g_k$$

Solve the Recurrence: The First Round

Recall: g_k is the good amplitude; $h_k = \sqrt{N-1}\, b_k$ combines the equal bad amplitudes. Choose $\alpha = \arcsin(1/\sqrt{N})$, so $g_0 = \sin\alpha$ and $h_0 = \cos\alpha$.

START WITH THE RECURRENCES FROM THE PREVIOUS PAGE

$$g_{k+1} = \cos 2\alpha\, g_k + \sin 2\alpha\, h_k, \qquad h_{k+1} = \cos 2\alpha\, h_k - \sin 2\alpha\, g_k$$

SUBSTITUTE THE INITIAL VALUES ($k = 0$)

$$g_1 = \cos 2\alpha \sin\alpha + \sin 2\alpha \cos\alpha = \sin(2\alpha + \alpha) = \sin 3\alpha$$

$$h_1 = \cos 2\alpha \cos\alpha - \sin 2\alpha \sin\alpha = \cos(2\alpha + \alpha) = \cos 3\alpha$$

We used $\sin(u+v) = \sin u \cos v + \cos u \sin v$ and $\cos(u+v) = \cos u \cos v - \sin u \sin v$.

From One Round to k Rounds

The same addition identities apply at every round. Suppose $g_k = \sin\beta_k$ and $h_k = \cos\beta_k$.

$$g_{k+1} = \cos 2\alpha \sin\beta_k + \sin 2\alpha \cos\beta_k = \sin(\beta_k + 2\alpha)$$

$$h_{k+1} = \cos 2\alpha \cos\beta_k - \sin 2\alpha \sin\beta_k = \cos(\beta_k + 2\alpha)$$

THE ARGUMENT INCREASES BY 2α EACH TIME

$$\beta_0 = \alpha \longrightarrow \beta_1 = 3\alpha \longrightarrow \beta_2 = 5\alpha \longrightarrow \beta_k = (2k+1)\alpha$$

$$g_k = \sin((2k+1)\alpha), \qquad b_k = \frac{h_k}{\sqrt{N-1}} = \frac{\cos((2k+1)\alpha)}{\sqrt{N-1}}$$

When Should We Stop?

Measurement succeeds with the squared good amplitude: $P_k = g_k^2 = \sin^2((2k+1)\alpha)$.

1. AIM FOR THE FIRST PEAK: $\sin^2(\pi/2) = 1$

$$(2k+1)\alpha \approx \frac{\pi}{2} \quad \Rightarrow \quad k \approx \frac{\pi}{4\alpha} - \frac{1}{2}$$

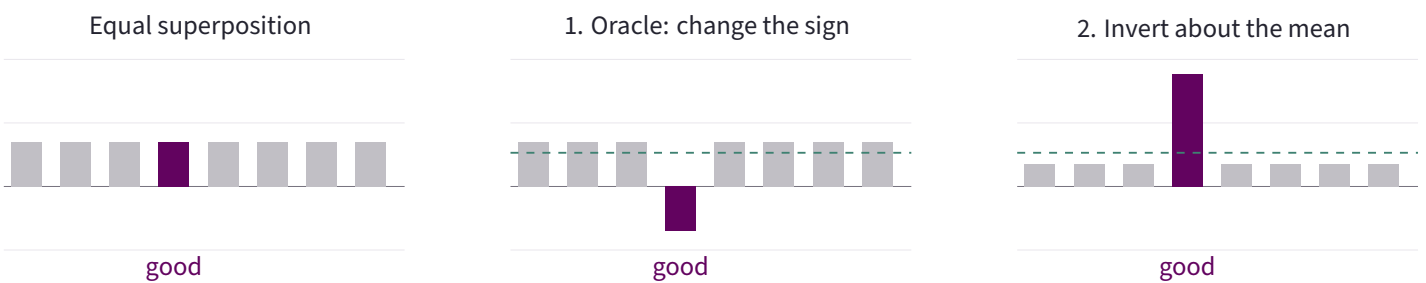
2. SUBSTITUTE $\alpha = \arcsin(1/\sqrt{N})$

$$k \approx \frac{\pi}{4\arcsin(1/\sqrt{N})} - \frac{1}{2} \approx \frac{\pi}{4}\sqrt{N} - \frac{1}{2} = O(\sqrt{N})$$

For large N, $\arcsin(1/\sqrt{N}) \approx 1/\sqrt{N}$. Choose the nearest integer k: the probability is close to one, not necessarily exactly one. More rounds can move past the peak.

The Oracle: Where the Problem Is Encoded

$$O_f |x\rangle = (-1)^{f(x)} |x\rangle , \qquad f(x) = 1 \text{ if } x \text{ is good}$$



The oracle tells the algorithm what counts as good. It encodes the problem condition and marks each satisfying component by changing its sign, as shown in the middle panel.

To apply Grover search to a new problem, design its oracle. The amplitude-amplification steps then increase the probability of measuring a marked answer.

Algorithm Notes

- Approximation algorithm? No: a bounded-error search algorithm.
  - With a suitable iteration count, the failure probability is at most $1/N$. A successful output is the exact good label.
- Parallel algorithm? It seems that the equal superposition state contains all of the possible states. When we use a rotation matrix to evolve the state, it is just like we are operating on all of the states **simultaneously**. This motivates different interpretations of the superposition: $|cat\rangle = |alive\rangle + |dead\rangle$
  - Opinion 1: superposition state is **one** state.
  - Opinion 2: superposition state contains **multiple** states.

These are interpretations; the calculation does not assume parallel universes.

Outline

MOON

CAT

EVOLVE

APPLE

Annealing

END

Annealing

From Search to Optimization

Two major approaches to quantum algorithms for data-management problems:

GROVER-BASED SEARCH

Examples: unstructured record search; finding a k-plex of a target size.

Mark candidates satisfying a condition.

QUANTUM ANNEALING (QA)

Examples: multiple query optimization; join-order optimization.

Minimize an encoded energy function.

Find x satisfying a condition $\longrightarrow$ Find x minimizing $E(x)$.

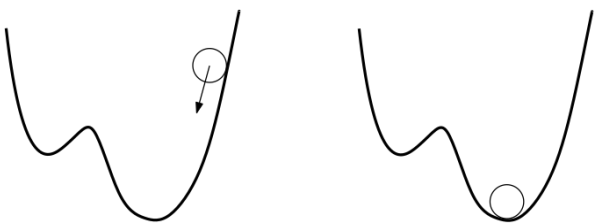
Encode condition violations as penalties: valid choices have zero penalty. Add a cost term when we also want the **best** valid choice, with weights that enforce validity.

Examples: Grover (1996); Li et al. (ICDE 2024); Trummer and Koch (PVLDB 2016); Join ordering (SIGMOD 2023).

Two Ways to Find a Low-Energy State

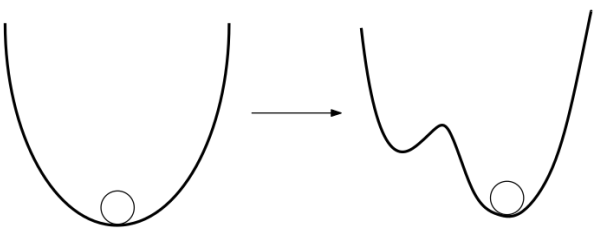
The landscape represents objective value (energy): lower valleys correspond to better solutions.

CLASSICAL SIMULATED ANNEALING



Explore a fixed landscape. Thermal uphill moves help escape local minima; gradually lower the temperature.

QUANTUM ANNEALING



Slowly reshape the landscape. Start in an easy lowest-energy state and evolve toward the problem's landscape.

Adiabatic theorem (ideal isolated system). Start in the lowest-energy state. If the landscape changes smoothly and slowly enough, with a nonzero gap to higher-energy states, the system stays close to its **changing lowest-energy state**.

How Quantum Annealing Evolves

H is the **Hamiltonian**: an operator describing the system's energy and evolution. A **ground state** has the lowest energy.

$$H(s) = A(s)H_{\text{driver}} + B(s)H_{\text{problem}}, \qquad 0 \leq s \leq 1$$

s is progress through the anneal; $A(s)$ and $B(s)$ control the strengths of the two terms.

H_{driver}: THE TRIVIAL TERM

An easy-to-control term, usually a transverse magnetic field. It enables quantum transitions between candidate bitstrings. Its ground state is easy to prepare: typically an equal superposition.

H_{problem}: THE TASK ENCODED AS ENERGY

It assigns each candidate x an energy $E(x)$: its cost plus constraint penalties. With suitable penalties, a lowest-energy state represents an **optimal valid answer**.

Prepare the driver's ground state $\longrightarrow$ reduce A, increase B $\longrightarrow$ **measure** a candidate solution.

Finite runs need not find the optimum. Reference: D-Wave, What is Quantum Annealing?

QUBO: A Common Optimization Form

Quadratic Unconstrained Binary Optimization

$$\min_{x_i \in \{0,1\}} E(x) = \sum_i a_i x_i + \sum_{i<j} b_{ij} x_i x_j + c$$

BINARY

x_i is a yes/no decision.
Example: build index i?

QUADRATIC

Degree **at most 2**: x_i, $x_i x_j$.
No cubic terms.

UNCONSTRAINED

Constraints are absorbed into penalty terms.

QUBO is a model, not an algorithm. Classical solvers can use it too.

Reference: D-Wave, QUBOs and Ising Models.

Example: Choose Exactly One Index

A fixed **workload** is a set of queries run repeatedly. We want to speed it up by building an **index**: an extra structure for locating data efficiently.

ILLUSTRATIVE ESTIMATES: ONE WORKLOAD RUN

Choice	Runtime	Saving
No new index	10 ms	0 ms
Index A only	7 ms	3 ms
Index B only	8 ms	2 ms

Deployment rule: add exactly one new index. Either candidate fits the storage budget, but both together do not.

DECISION AND OPTIMIZATION MODEL

$I = \{A, B\}$; for each $i \in I$,
$x_i = 1$ means build i; $x_i = 0$ means do not.

$$\max_{x_A, x_B} \quad B(x_A, x_B) = 3x_A + 2x_B$$

$$\text{subject to} \quad x_A + x_B = 1,$$

$$x_A, x_B \in \{0, 1\}.$$

Only $(1, 0)$ and $(0, 1)$ are feasible. Neither or both violates the deployment rule.

For QUBO, minimize $-B$. Without the constraint, the formula favors $(1, 1)$ with -5; next, we penalize this invalid choice.

Turn the Constraint into a Penalty

The constraint is $x_A + x_B = 1$. Move everything to the left, square the violation, and multiply by a positive weight λ: $P = \lambda(x_A + x_B - 1)^2$.

$$E(x_A, x_B) = -3x_A - 2x_B + \lambda(x_A + x_B - 1)^2$$

2. CHECK WHAT THE SQUARE DOES

Choice	$x_A + x_B$	Violation	P
Neither	0	-1	λ
Exactly one	1	0	0
Both	2	1	λ

WITH $\lambda = 4$: COMBINE BOTH TERMS

$$E = -3x_A - 2x_B + 4(x_A + x_B - 1)^2$$

$E = 4 - 7x_A - 6x_B + 8x_Ax_B$ since $x_i^2 = x_i$.

3. How should we choose λ?

Think for 1 minute

Find a condition on λ that guarantees every minimum-energy solution chooses **exactly one index**.

Is any positive weight sufficient? Explain why the choice $\lambda = 4$ on the left works.

Turn the Constraint into a Penalty

The constraint is $x_A + x_B = 1$. Move everything to the left, square the violation, and multiply by a positive weight λ: $P = \lambda(x_A + x_B - 1)^2$.

$$E(x_A, x_B) = -3x_A - 2x_B + \lambda(x_A + x_B - 1)^2$$

2. CHECK WHAT THE SQUARE DOES

Choice	$x_A + x_B$	Violation	P
Neither	0	-1	λ
Exactly one	1	0	0
Both	2	1	λ

WITH $\lambda = 4$: COMBINE BOTH TERMS

$$E = -3x_A - 2x_B + 4(x_A + x_B - 1)^2$$

$E = 4 - 7x_A - 6x_B + 8x_Ax_B$ since $x_i^2 = x_i$.

3. MAKE THE INVALID CHOICE WORSE

A only (valid): $E_A = -3 + 0 = -3$.

Both (invalid): $E_{AB} = -5 + \lambda$.

Adding B improves the objective term by 2, but violates the constraint. The penalty must **outweigh this improvement**:

$$-5 + \lambda > -3 \quad \Longrightarrow \quad \lambda > 2.$$

$\lambda = 2$ ties: the invalid choice is also optimal.
Choose $\lambda = 4$: $E_{AB} = -1 > -3 = E_A$.

Check the QUBO and Read the Answer

Evaluate all four choices: $E = -3x_A - 2x_B + 4(x_A + x_B - 1)^2$. Lower is better.

x_A	x_B	Benefit	Penalty	Energy
0	0	0	4	4
1	0	3	0	$\mathbf{-3}$ (minimum)
0	1	2	0	-2
1	1	5	4	-1

RESULT: BUILD INDEX A

The penalty prevents choosing both. Here weight 4 is sufficient: both scores -1, worse than the feasible optimum -3.

A solver returns candidate bitstrings. Decode each one, check feasibility, and compare energies.

This table proves the toy optimum. Real annealing runs sample candidates; they need not return the optimum every time.

Quantum Multiple Query Optimization (MQO)

A **classic database problem**: choose **one execution plan for each query** so that answering all queries together has the lowest total cost.

INPUT: PLANS, COSTS, AND SHARING

Queries Q, each with candidate plans P_q. These plans compute the **same query result in different ways**, e.g., different join orders.

c_p: estimated cost of running plan p alone.

$s_{p,p'}$: cost saved when two selected plans reuse intermediate results (pairwise sharing).

EXAMPLE: TWO QUERIES

$q_1 : p_1,\ \boldsymbol{p_2}$

$q_2 : \boldsymbol{p_3},\ p_4$

$P_e = \{p_2, p_3\}$ is **valid**: one plan per query. Costs and savings determine whether it is **optimal**.

Goal: among all valid selections, minimize total plan costs − sharing savings.

Quantum case study: Trummer and Koch, *PVLDB* 9(9):648–659, 2016 (VLDB 2016). MQO → QUBO → D-Wave 2X.

Why the Cheapest Individual Plans Can Lose

Two queries, two plans each. Costs are estimated cost units, not measured runtimes.

INPUT: PLAN COSTS

q_1 : $p_1 : 2$ $p_2 : 4$

q_2 : $p_3 : 3$ $p_4 : 1$

Sharing saving: $s_{p_2,p_3} = 5$.

Only when both plans are selected. All other pairwise savings are zero.

COMPARE ALL VALID CHOICES

Plans	Sharing	Total cost
p_1, p_3	0	$2 + 3 = 5$
p_1, p_4	0	$2 + 1 = 3$
p_2, p_3	5	$\mathbf{4 + 3 - 5 = 2}$
p_2, p_4	0	$4 + 1 = 5$

Separate choices pick p_1, p_4 (cost 3). Joint optimization picks p_2, p_3 (cost 2).

Build the QUBO: Pay for Plans, Subtract Savings

Use one binary variable per plan: $x_i = 1$ selects p_i; $x_i = 0$ does not.

1. PAY FOR EACH SELECTED PLAN

$$E_C = 2x_1 + 4x_2 + 3x_3 + x_4$$

Each selected plan adds its cost. An unselected plan contributes zero.

2. CREDIT SHARING ONLY WHEN BOTH RUN

$$E_S = -5x_2x_3$$

$x_2x_3 = 1$ only when both plans are selected. Otherwise the saving is zero.

What is missing? Minimizing $E_C + E_S$ alone chooses no plans: energy 0, lower than the best valid cost 2. It has saved money by answering **no queries**.

Enforce Exactly One Plan per Query

$q_1 : p_1, \boldsymbol{p_2}$ $\quad q_2 : \boldsymbol{p_3}, p_4$ $\quad P_e = \{p_2, p_3\}$ is valid: one plan per query.

ENCOURAGE SELECTION

$$-w_L \sum_i x_i, \qquad w_L > 0$$

Each selected plan earns an energy reduction w_L. This discourages leaving a query unanswered, but also rewards selecting extra plans.

PENALIZE MULTIPLE SELECTIONS

$$w_M(x_1x_2 + x_3x_4), \qquad w_M > 0$$

Charge w_M when both alternatives for the **same query** are selected. Sharing between different queries remains allowed.

For one query	No plan	One plan	Both plans
Constraint contribution	0	$-w_L$	$-2w_L + w_M$

Neither term works alone. Together, with suitable weights, they make exactly one selection preferable **even after execution costs and sharing savings are included**.

The Complete Energy Function

$$E(x) = \underbrace{2x_1 + 4x_2 + 3x_3 + x_4 - 5x_2x_3}_{\text{execution cost minus sharing savings}}$$

$$-w_L(x_1 + x_2 + x_3 + x_4) + w_M(x_1x_2 + x_3x_4).$$

Next question: how should we choose the weights w_L and w_M?

Penalty Weights: Strong Enough, Not Arbitrarily Huge

$$E(x) = \underbrace{2x_1 + 4x_2 + 3x_3 + x_4 - 5x_2x_3}_{\text{execution cost minus sharing savings}} - w_L(x_1 + x_2 + x_3 + x_4) + w_M(x_1x_2 + x_3x_4).$$

TOO SMALL: AN INVALID ANSWER MAY WIN

If $w_L = 0$, selecting nothing has energy 0; the best valid choice costs 2. Answering no queries can win.

If $w_L = 5$ but $w_M = 1$, selecting all four plans gives $E = 10 - 5 - 20 + 2 = -13$. The best valid choice gives $2 - 10 = -8$: the invalid choice wins.

TOO LARGE: VALID, BUT SLOW TO IMPROVE

With $w_L = 1000,\ w_M = 2000$, $\{p_1, p_4\}$ is valid (cost 3), but $\{p_2, p_3\}$ is optimal (cost 2).

To switch $p_1 \to p_2$ one bit at a time: remove p_1 ($\Delta E = 998$) or add p_2 ($\Delta E = 1004$). Either first step is invalid.

High barriers can slow convergence, leaving a finite run with a **valid but suboptimal** solution.

Single-bit illustration; quantum transitions can be collective.

Design principle: bound the largest incentive to violate a constraint, then exceed it by a modest margin.

Homework 2: How Should We Choose the Weights for QMQO

For any instance, assume $c_p \geq 0$ and pairwise savings $s_{pr} \geq 0$. Let P_q be the alternative plans for query q; $x_p = 1$ means select plan p.

$$E = \sum_p c_p x_p - \sum_{\{p,r\}} s_{pr} x_p x_r - w_L \sum_p x_p + w_M \sum_q \sum_{\{p,r\} \subseteq P_q} x_p x_r$$

How should we choose w_L and w_M so that **every minimum-energy solution** selects **exactly one plan for each query**?

Derive sufficient conditions and justify your answer.

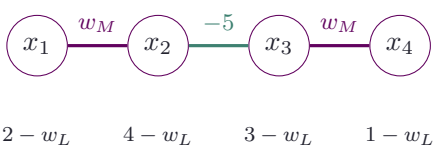
Due: one week before the final exam.

QA Stage 2: Physical Mapping

$$E(x) = \underbrace{2x_1 + 4x_2 + 3x_3 + x_4 - 5x_2x_3}_{\text{execution cost minus sharing savings}} - w_L(x_1 + x_2 + x_3 + x_4) + w_M(x_1x_2 + x_3x_4).$$

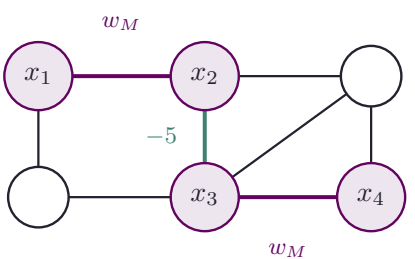
Place variables on physical qubits. Solid edges show available **couplers**; each pair term needs a connection.

LOGICAL QUBO



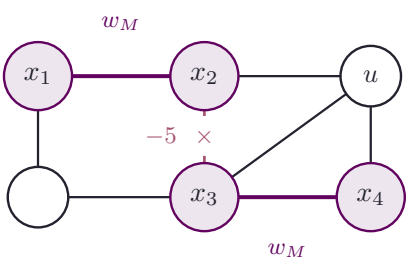
Below nodes: linear terms.
Edges: pair terms.

DIRECT FIT



Every required interaction has a direct coupler.

MISSING DIRECT LINK



Dashed: no direct x_2–x_3 link. Route via spare u.

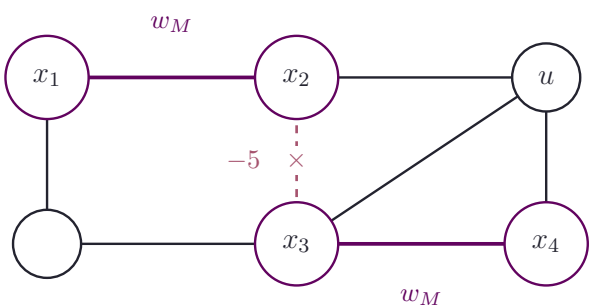
The right-hand placement needs a **qubit chain** along the available indirect path (next).

Illustrative hardware. QUBO weights are shown; convert them to Ising biases and couplings before programming.

Physical Mapping: Repair the Placement with a Chain

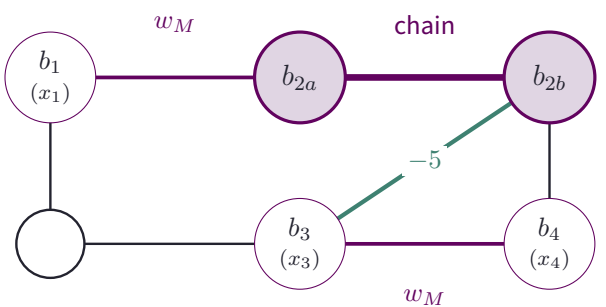
Use the **same hardware and placement** as the failed example. The spare qubit u provides a route from x_2 to x_3.

BEFORE: THE DIRECT COUPLER IS MISSING



Keep other assignments fixed. Use the available path $x_2 \to u \to x_3$.

AFTER: TWO PHYSICAL BITS FOR x_2



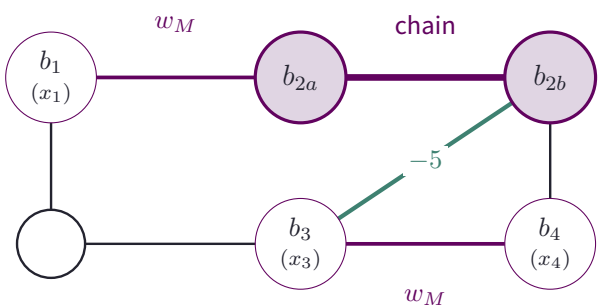
Enforce $b_{2a} = b_{2b} = x_2$. The diagonal coupler now implements $-5b_{2b}b_3 = -5x_2x_3$.

Another direct placement may work. This example repairs the placement shown; the following pages explain the recovered graph and chain energy.

Physical Mapping: Recover the Logical Graph

For an agreeing chain, define $b_2 = b_{2a} = b_{2b}$. The contracted graph uses b_1, b_2, b_3, b_4, with $b_i = x_i$ for $i = 1, \ldots, 4$.

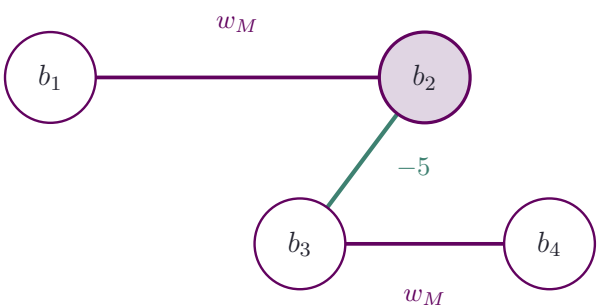
PHYSICAL QUBITS: SAME AS PREVIOUS PAGE



Two physical bits store the **same** value $b_2 = x_2$. This does not create a new physical qubit.

merge

CONTRACTED GRAPH IN b VARIABLES



Their linear weights add together; the chain penalty is zero when they agree.

The recovered graph shows nonzero programmed pair terms only. Unused physical couplers have weight zero.

Make a Qubit Chain Behave Like One Bit

$$E(x) = 2x_1 + 4x_2 + 3x_3 + x_4 - 5x_2x_3 - w_L(x_1 + x_2 + x_3 + x_4) + w_M(x_1x_2 + x_3x_4).$$

w_L, w_M: the original query-constraint weights. $K > 0$: the new **chain penalty weight**.

PENALIZE DISAGREEMENT

$$P_{\text{chain}} = K(b_{2a} - b_{2b})^2$$

Broken chain: unequal bits, no single logical value.

b_{2a}	b_{2b}	Penalty
0	0	0
1	1	0
0	1	K
1	0	K

THE SAME EXAMPLE ON PHYSICAL QUBITS

Split x_2's coefficient $4 - w_L$ equally between b_{2a} and b_{2b}.

$$\begin{aligned} E_{\text{phys}} = {} & (2 - w_L)b_1 \\ & + \tfrac{4-w_L}{2}(b_{2a} + b_{2b}) \\ & + (3 - w_L)b_3 + (1 - w_L)b_4 \\ & + w_M b_1 b_{2a} - 5b_{2b}b_3 + w_M b_3 b_4 \\ & + K(b_{2a} - b_{2b})^2. \end{aligned}$$

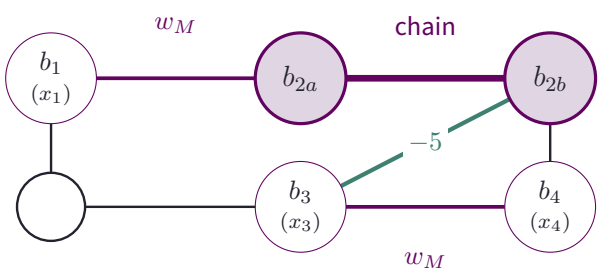
If $b_{2a} = b_{2b} = x_2$, then $E_{\text{phys}} = E(x)$. Choose K large enough that breaking the chain cannot pay off.

Fewer Cross Terms Can Mean Fewer Physical Qubits



WITH $-5x_2x_3$: 3 CROSS TERMS

$$E_{\text{with}}(x) = 2x_1 + 4x_2 + 3x_3 + x_4 \\ -5x_2x_3 - w_L(x_1 + x_2 + x_3 + x_4) \\ + w_M(x_1x_2 + x_3x_4).$$

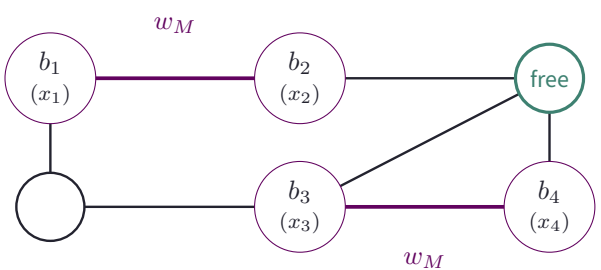


x_2: a **two-qubit chain**

5 physical qubits in use

WITHOUT $-5x_2x_3$: 2 CROSS TERMS

$$E_{\text{without}}(x) = 2x_1 + 4x_2 + 3x_3 + x_4 \\ - w_L(x_1 + x_2 + x_3 + x_4) \\ + w_M(x_1x_2 + x_3x_4).$$

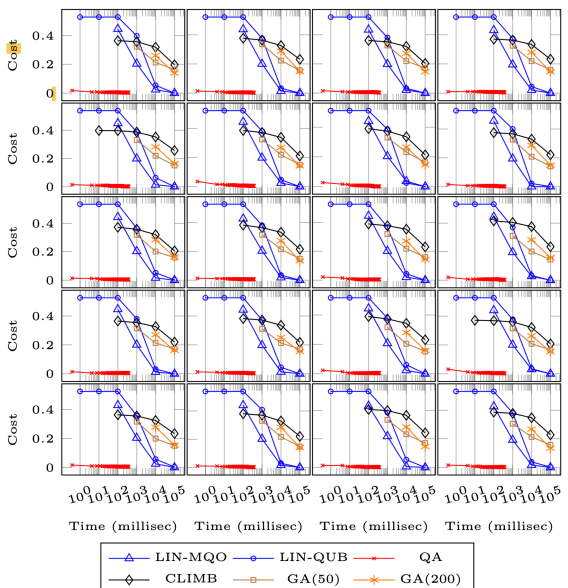


x_2: **one qubit; no chain**

4 physical qubits in use

Same hardware and placement; black couplers have weight zero. Removing the term changes the objective.

Experiment: Solution Quality vs. Optimization Time



HOW TO READ THE FIGURE

20 panels: 20 MQO instances, each with 537 queries and two plans per query.

Horizontal: optimization time (ms, logarithmic). Vertical: scaled cost of the selected plans; lower is better. Red: quantum annealing (QA).

QA reaches low-cost solutions early on these instances. The other curves show integer programming, hill climbing, and genetic algorithms.

The plot excludes preprocessing/embedding time; it compares optimization time, not end-to-end database query runtime.

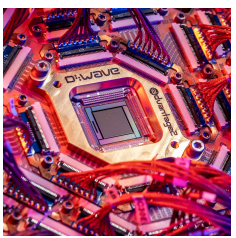
Source: Trummer and Koch, PVLDB 9(9), 2016, Figure 4. The paper reports 112–135 ms of preprocessing per test case.

Current Hardware: Annealing and Gate-Based Computing

Latest deployed generations (October 2026): two ways to program a quantum computer.

D-Wave Advantage2

QUANTUM ANNEALING



4,400+

physical qubits

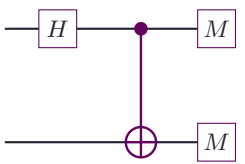
Up to 20 neighbors
40,000+ couplers

About $20\,\mathrm{mK}$; Leap cloud or on-site access.

Program an energy function.
Annealing searches for low-energy solutions, as in our MQO example.

IBM Nighthawk r2

GATE-BASED QUANTUM COMPUTING



Example circuit

120

programmable qubits

218 couplers
Up to 4 neighbors

r2 (Sep 2026): fast reset between circuit runs.

Program gates, then measure (M).
Grover's oracle and diffusion are implemented as quantum circuits.

Sources: D-Wave Advantage2; IBM Nighthawk r2; IBM processor types. Different computing models; qubit counts alone do not compare capability.

Outline

MOON

CAT

EVOLVE

APPLE

Annealing

END

END

When you are not looking at the moon, is it there?



We don’t know